МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 5 - 6

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Шерри Георгий Даниэлевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

ОБЪЕКТЫ БАЗЫ ДАННЫХ: ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ, ХРАНИМЫЕ ПРОЦЕДУРЫ И ТРАНЗАКЦИИ

Цель работы: Работа направлена на формирование у студентов углубленных навыков работы с объектами баз данных в СУБД PostgreSQL, смещая акцент от прямого манипулирования данными к созданию переиспользуемых логических конструкций и безопасному управлению транзакциями.

Ниже приводятся таблицы, используемые для построения запросов.

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	full_name			passport_data			email	login	password
		last_name	first_name	middle_name	series	number	issued_by	issue_date			
1	1	Иванов	Иван	Иванович	4510	123456	УФМС России по г. Москве	2015-03-10	ivanov@mail.ru	ivan_i	pass123
2	2	Петрова	Анна	Сергеевна	5012	789012	МВД по Республике Татарст	2018-11-20	anna.petrova@list	annap	qwerty
3	3	Сидоров	Алексей	Владимирович	4612	345678	УФМС по Ростовской облас	2020-01-25	sid_alex@yandex.r	alexs	1q2w3e
4	4	Козлова	Елена	Дмитриевна	4711	987654	Отдел УФМС по г. Санкт-Пет	2019-06-30	lenakozlova@gma	lenak	zaq12wsx
5	5	Морозов	Дмитрий	Алексеевич	4522	556677	УФМС по Краснодарскому к	2021-02-14	morozov_d@bk.ru	dmorozov	pass321

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	name	country_id	city_id	stars	full_address					
						postal_code	country	city	street	building	apartment
1	1	Sunset Beach Resort	2	3	5	07100	Турция	Анталья	Lara Beach	Bina Yolu No:15	[NULL]
2	2	Grand Whale	1	1	4	141008	Россия	Москва	Ленинградское ш	37	1B
3	3	Jaz Aquamarine	3	5	5	84511	Египет	Хургада	Safaga Road	1	[NULL]
4	4	Green Park Resort	4	8	4	83150	Таиланд	Пхукет	Patong Beach Roa	133	[NULL]
5	5	Atlantis The Palm	5	9	5	00000	ОАЭ	Дубай	Palm Jumeirah	1	[NULL]

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	created_at	client_id	employee_id	status_id	paid_at	total_amount
1	1	2025-01-10 10:15:00.000 +0300	1	2	5	2025-01-10 12:00:00.000 +0300	85 000
2	2	2025-01-15 14:30:00.000 +0300	2	4	4	[NULL]	120 000
3	3	2025-01-20 09:45:00.000 +0300	3	5	1	[NULL]	45 000
4	4	2025-02-05 11:20:00.000 +0300	4	6	2	[NULL]	210 000
5	5	2025-02-10 16:00:00.000 +0300	5	2	6	2025-02-11 10:00:00.000 +0300	67 000

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

	service_id	order_id	quantity	price
1	1	1	1	85 000
2	3	2	2	60 000
3	5	3	1	45 000
4	2	4	1	210 000
5	4	5	1	67 000

Задание №1: разработка пользовательских функций

Листинг 1

```
CREATE OR REPLACE FUNCTION get_client_total_spent(p_client_id INT)
RETURNS NUMERIC(10,2) AS $$
DECLARE
    total_spent NUMERIC(10,2);
    client_exists BOOLEAN;
BEGIN
    -- Проверка существования клиента
    SELECT EXISTS(SELECT 1 FROM client WHERE id = p_client_id) INTO
client_exists;

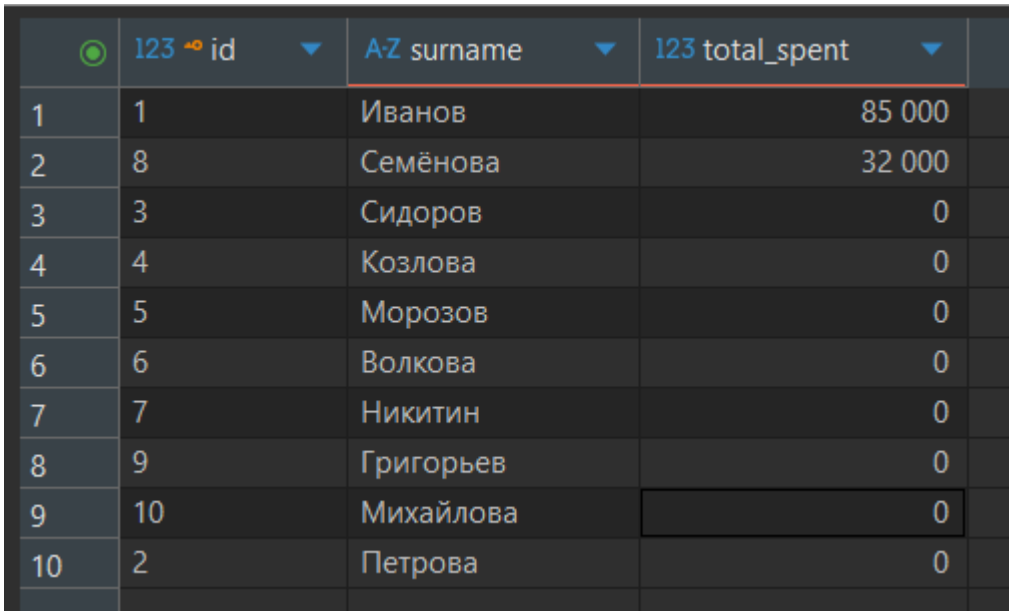
    IF NOT client_exists THEN
        RAISE EXCEPTION 'Клиент с ID % не найден.', p_client_id;
    END IF;

    -- Подсчет суммы (только для заказов со статусом "Оплачен полностью")
    SELECT COALESCE(SUM(total_amount), 0.00) INTO total_spent
    FROM customer_order
    WHERE client_id = p_client_id
      AND status_id = (SELECT id FROM order_status WHERE name = 'Оплачен
полностью');

    RETURN total_spent;
END;
$$ LANGUAGE plpgsql;

-- Вызов функции:
SELECT id, (full_name).last_name AS surname, get_client_total_spent(id) AS
total_spent
FROM client
ORDER BY total_spent DESC;
```

Name	Value
Updated Rows	0
Execute time	0,003s
Start time	Sat Oct 03 19:20:31 MSK 2026
Finish time	Sat Oct 03 19:20:32 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_client_total_spent(p_client_id INT) RETURNS NUMERIC(10,2) AS $$ DECLARE total_spent NUMERIC(10,2); client_exists BOOLEAN; BEGIN -- Проверка существования клиента SELECT EXISTS(SELECT 1 FROM client WHERE id = p_client_id) INTO client_exists; IF NOT client_exists THEN RAISE EXCEPTION 'Клиент с ID % не найден.', p_client_id; END IF; -- Подсчет суммы (только для заказов со статусом "Оплачен полностью") SELECT COALESCE(SUM(total_amount), 0.00) INTO total_spent FROM customer_order WHERE client_id = p_client_id AND status_id = (SELECT id FROM order_status WHERE name = 'Оплачен полностью'); RETURN total_spent; END; $$ LANGUAGE plpgsql



	123 id	A-Z surname	123 total_spent
1	1	Иванов	85 000
2	8	Семёнова	32 000
3	3	Сидоров	0
4	4	Козлова	0
5	5	Морозов	0
6	6	Волкова	0
7	7	Никитин	0
8	9	Григорьев	0
9	10	Михайлова	0
10	2	Петрова	0

Листинг 2

```
CREATE OR REPLACE FUNCTION get_hotel_info(
    p_hotel_id INT,
    OUT h_name VARCHAR,
    OUT h_stars SMALLINT,
    OUT h_country VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT h.name, h.stars, c.name
    INTO h_name, h_stars, h_country
    FROM hotel h
    JOIN country c ON h.country_id = c.id
    WHERE h.id = p_hotel_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Отель с ID % не найден.', p_hotel_id;
    END IF;
END;
$$;

-- Вызов функции:
SELECT * FROM get_hotel_info(1);
```

Статистика 1

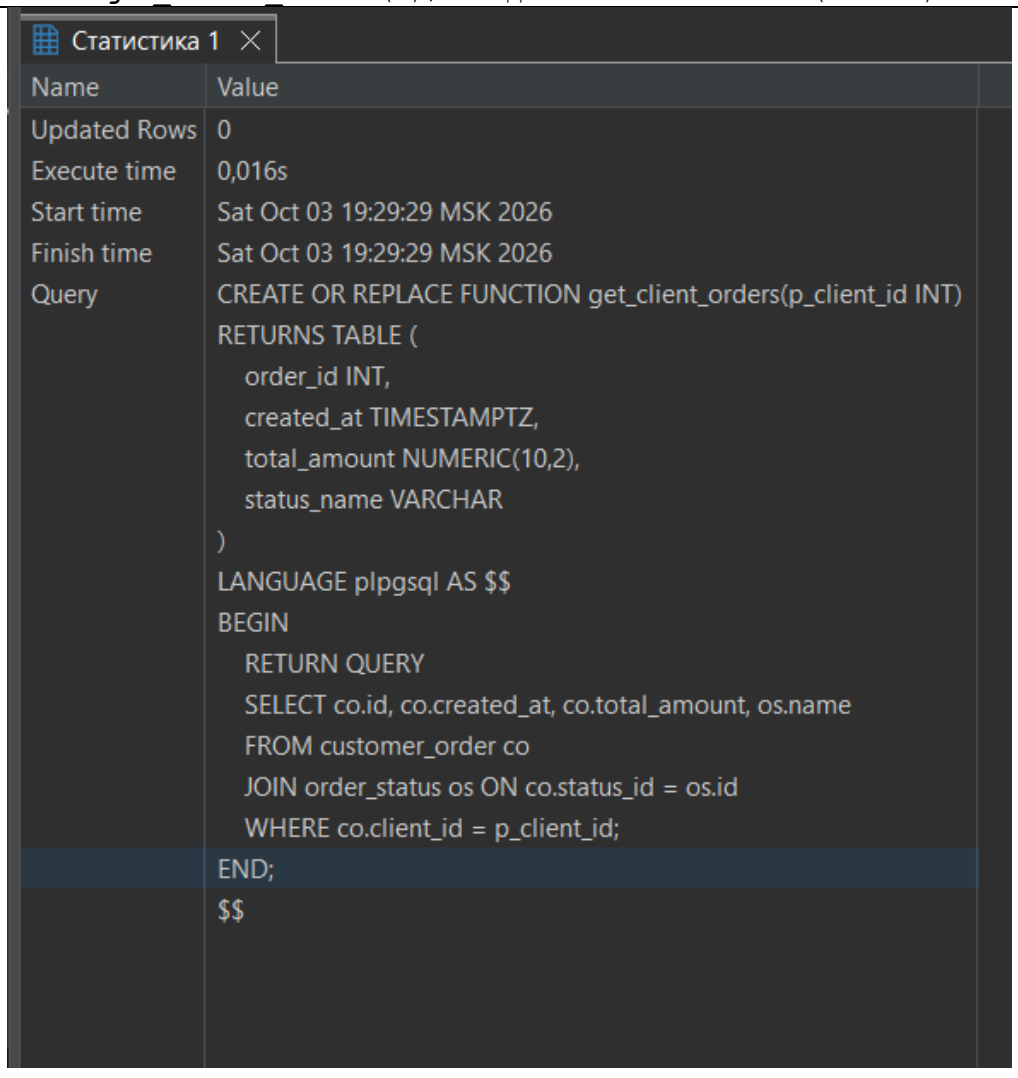
Name	Value
Updated Rows	0
Execute time	0,004s
Start time	Sat Oct 03 19:27:28 MSK 2026
Finish time	Sat Oct 03 19:27:28 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_hotel_info(p_hotel_id INT, OUT h_name VARCHAR, OUT h_stars SMALLINT, OUT h_country VARCHAR) LANGUAGE plpgsql AS $$ BEGIN SELECT h.name, h.stars, c.name INTO h_name, h_stars, h_country FROM hotel h JOIN country c ON h.country_id = c.id WHERE h.id = p_hotel_id; IF NOT FOUND THEN RAISE EXCEPTION 'Отель с ID % не найден.', p_hotel_id; END IF; END; $$

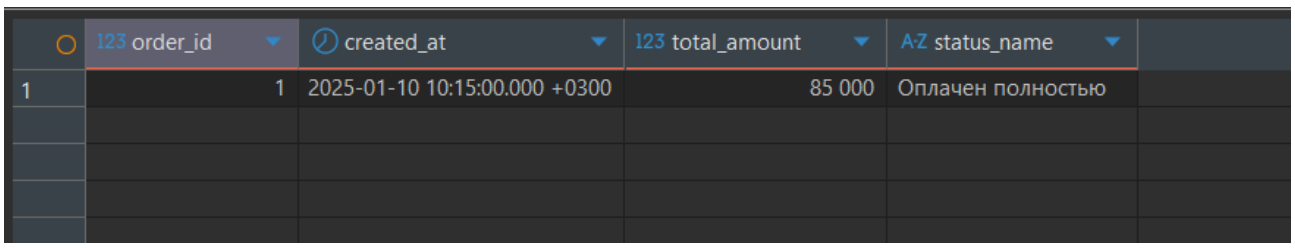
	A-Z h_name	123 h_stars	A-Z h_country
1	Sunset Beach Resort	5	Турция

Листинг 3

```
CREATE OR REPLACE FUNCTION get_client_orders(p_client_id INT)
RETURNS TABLE (
    order_id INT,
    created_at TIMESTAMPTZ,
    total_amount NUMERIC(10,2),
    status_name VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT co.id, co.created_at, co.total_amount, os.name
    FROM customer_order co
    JOIN order_status os ON co.status_id = os.id
    WHERE co.client_id = p_client_id;
END;
$$;

-- Вызов функции:
SELECT * FROM get_client_orders(1); -- Для клиента с id=1 (Иванов)
```





Задание №2: разработка хранимой процедуры с управлением транзакциями

Листинг 4

```
CREATE OR REPLACE PROCEDURE process_order_payment(
    p_order_id INT,
    p_payment_amount NUMERIC(10,2),
    p_payment_type_id INT,
    OUT p_success BOOLEAN,
    OUT p_message TEXT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_order_total NUMERIC(10,2);
    v_order_status INT;
    v_paid_status_id INT;
BEGIN
    -- Получаем ID статуса "Оплачен полностью"
    SELECT id INTO v_paid_status_id FROM order_status WHERE name = 'Оплачен
полностью';

    -- Блокируем строку заказа (FOR UPDATE) для предотвращения состояния гонки
    SELECT total_amount, status_id INTO v_order_total, v_order_status
    FROM customer_order
    WHERE id = p_order_id
    FOR UPDATE;

    -- Проверка 1: Заказ существует?
    IF NOT FOUND THEN
        p_success := FALSE;
        p_message := 'Ошибка: Заказ не найден.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Проверка 2: Заказ не отменен?
    IF v_order_status IN ((SELECT id FROM order_status WHERE name IN ('Отменён
клиентом', 'Отменён поставщиком'))) THEN
        p_success := FALSE;
        p_message := 'Ошибка: Заказ отменен, оплата невозможна.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Проверка 3: Сумма оплаты корректна?
    IF p_payment_amount <= 0 OR p_payment_amount > v_order_total THEN
        p_success := FALSE;
        p_message := 'Ошибка: Некорректная сумма оплаты.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Успешные DML-операции
    INSERT INTO payment (order_id, amount, status_id, payment_type_id)
    VALUES (p_order_id, p_payment_amount, v_paid_status_id,
p_payment_type_id);

    UPDATE customer_order
    SET paid_at = CURRENT_TIMESTAMP,
        status_id = v_paid_status_id
    WHERE id = p_order_id;
```

```
    -- Фиксация транзакции
    COMMIT;
    p_success := TRUE;
    p_message := 'Оплата успешно проведена.';
END;
$$;
```

Name	Value
Updated Rows	0
Execute time	0,006s
Start time	Sat Oct 03 19:37:46 MSK 2026
Finish time	Sat Oct 03 19:37:46 MSK 2026

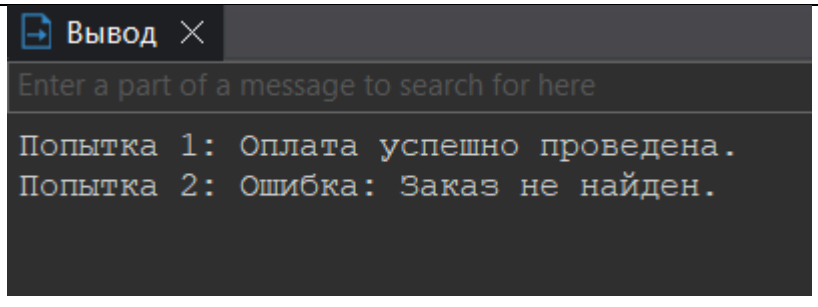
Query	CREATE OR REPLACE PROCEDURE process_order_payment(
	p_order_id INT,
	p_payment_amount NUMERIC(10,2),
	p_payment_type_id INT,
	OUT p_success BOOLEAN,
	OUT p_message TEXT
	)
	LANGUAGE plpgsql AS $$
	DECLARE
	v_order_total NUMERIC(10,2);
	v_order_status INT;
	v_paid_status_id INT;
	BEGIN
	-- Получаем ID статуса "Оплачен полностью"
	SELECT id INTO v_paid_status_id FROM order_status WHERE name = 'Оплачен полностью';
	-- Блокируем строку заказа (FOR UPDATE) для предотвращения состояния гонки
	SELECT total_amount, status_id INTO v_order_total, v_order_status
	FROM customer_order
	WHERE id = p_order_id
	FOR UPDATE;
	-- Проверка 1: Заказ существует?
	IF NOT FOUND THEN
	p_success := FALSE;

Демонстрация вызова (успешный и неудачный):

Листинг 5

```
DO $$
DECLARE
    v_success BOOLEAN;
    v_message TEXT;
BEGIN
    -- Попытка 1: Успешная оплата (Заказ id=3, сумма 45000, тип оплаты СБП
id=3)
    CALL process_order_payment(3, 45000.00, 3, v_success, v_message);
    RAISE NOTICE 'Попытка 1: %', v_message;

    -- Попытка 2: Неудачная (Заказ не существует)
    CALL process_order_payment(9999, 1000.00, 1, v_success, v_message);
    RAISE NOTICE 'Попытка 2: %', v_message;
END;
$$;
```



Задание №3: демонстрация атомарной транзакции (COMMIT)

Обновить email клиента и добавить ему новый телефон. Обе операции должны выполниться вместе.

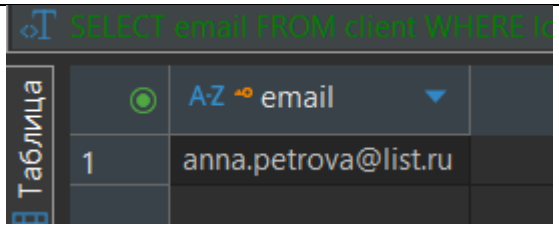
Листинг 6

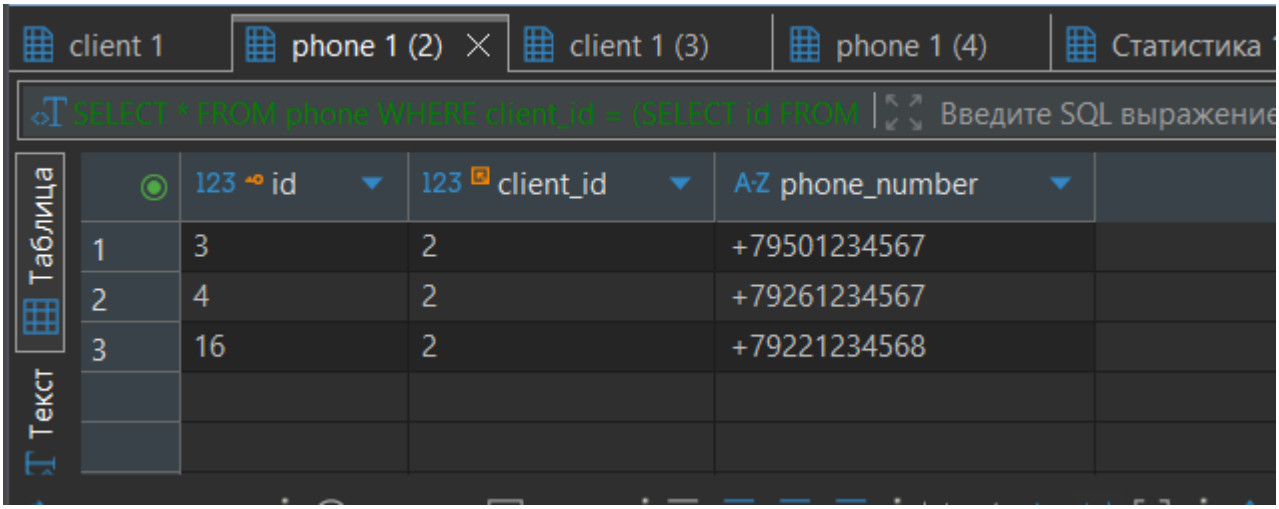
```
-- Шаг 1. Анализ "ДО"
SELECT email FROM client WHERE login = 'annap';
SELECT * FROM phone WHERE client_id = (SELECT id FROM client WHERE login =
'annap');

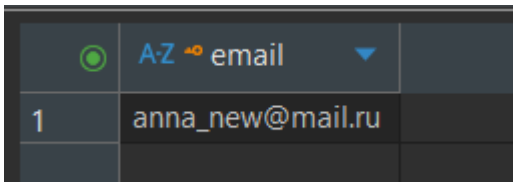
-- Шаг 2. Выполняем атомарную операцию
BEGIN;
    UPDATE client
    SET email = 'anna_new@mail.ru'
    WHERE login = 'annap';

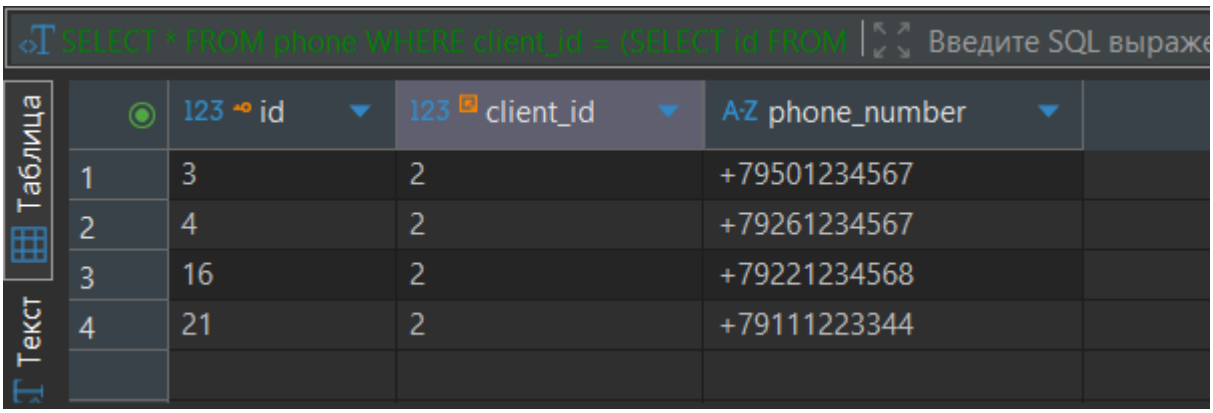
    INSERT INTO phone (client_id, phone_number)
    VALUES ((SELECT id FROM client WHERE login = 'annap'), '+79111223344');
COMMIT;

-- Шаг 3. Анализ "ПОСЛЕ"
SELECT email FROM client WHERE login = 'annap';
SELECT * FROM phone WHERE client_id = (SELECT id FROM client WHERE login =
'annap');
```









Задание №4: демонстрация отката транзакции (ROLLBACK)

Листинг 7

```
-- Ошибка возникнет из-за того, что id = 1 уже существует в таблице country
(это Россия).
BEGIN;
    -- 1. Успешное обновление (звезды отеля изменятся, но пока только внутри
транзакции)
    UPDATE hotel SET stars = 5 WHERE id = 10;

    -- 2. Намеренная ошибка: нарушение PRIMARY KEY (id=1 уже есть)
    INSERT INTO country (id, name, code)
    VALUES (1, 'Тестовая страна', 'TST');

COMMIT;

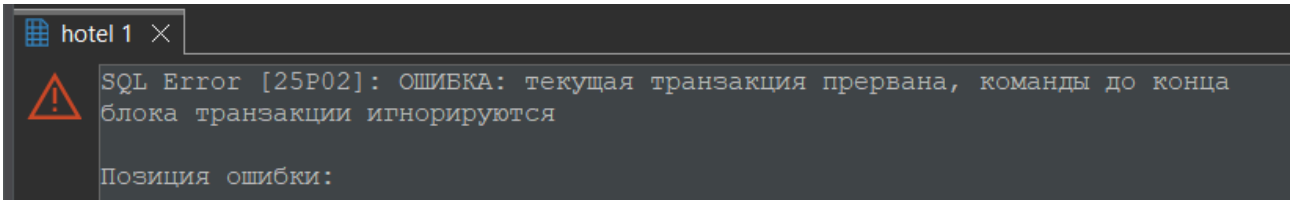

SELECT stars FROM hotel WHERE id = 10;

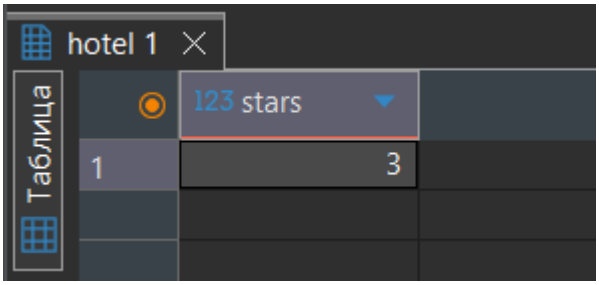
ROLLBACK;

SELECT stars FROM hotel WHERE id = 10;
```

```
SQL Error [23505]: ОШИБКА: повторяющееся значение ключа нарушает ограничение
уникальности "country_pkey"
  Подробности: Ключ "(id)=(1)" уже существует.

Позиция ошибки:
```





Задание №5: моделирование трёх аномалий параллельного доступа

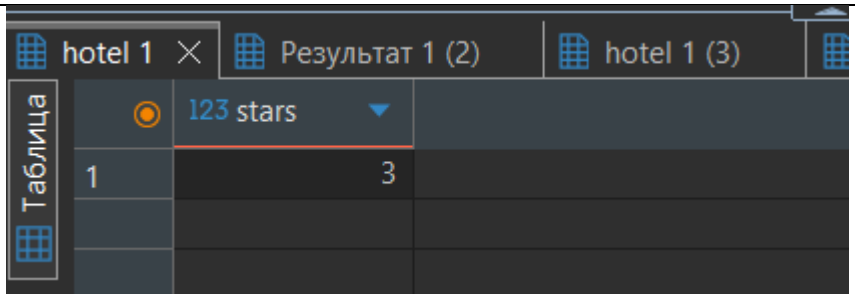
Моделирование аномалии «Неповторяемое чтение»

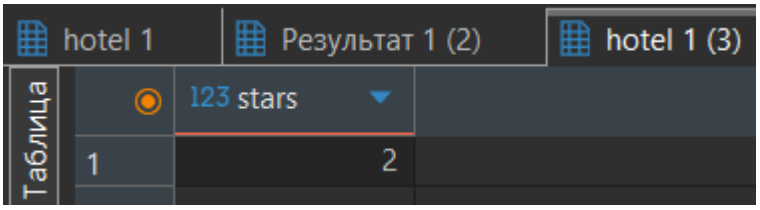
Листинг 8 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SELECT stars FROM hotel WHERE id = 10; -- Первое чтение
SELECT pg_sleep(10);
SELECT stars FROM hotel WHERE id = 10; -- Второе чтение
COMMIT;
```

Листинг 9 Скрипт Б, для выполнения во втором окне.

```
UPDATE hotel SET stars = 2 WHERE id = 10;
COMMIT;
```





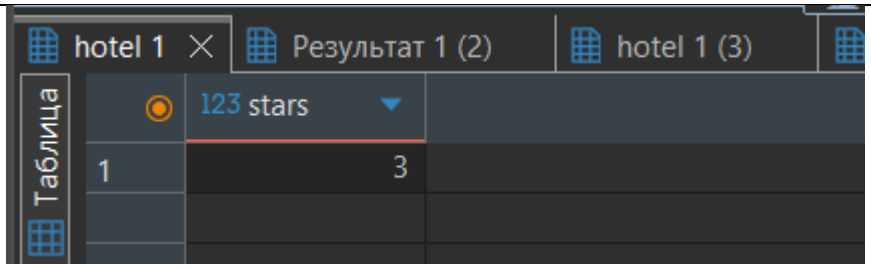
Устранение аномалии «Неповторяемое чтение»

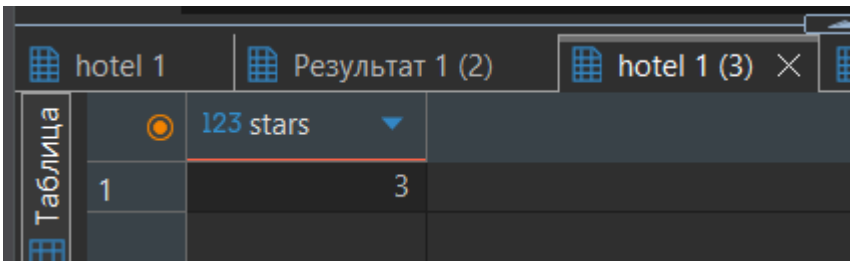
Листинг 10 Скрипт А, для выполнения в первом окне (обновлённая версия).

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stars FROM hotel WHERE id = 10;
SELECT pg_sleep(10);
SELECT stars FROM hotel WHERE id = 10;
COMMIT;
```

Листинг 11 Скрипт Б, для выполнения во втором окне (БЕЗ ИЗМЕНЕНИЙ).

```
UPDATE hotel SET stars = 2 WHERE id = 10;
COMMIT;
```





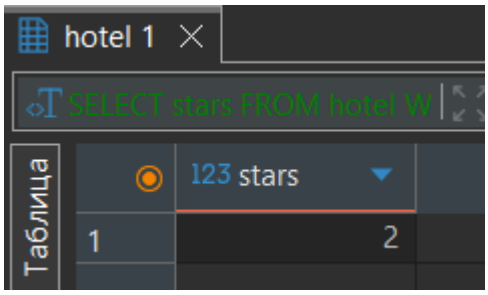
Моделирование аномалии «Грязное чтение» (Dirty Read)

Листинг 12 . Скрипт А (Окно 1)

```
BEGIN;
UPDATE hotel SET stars = 1 WHERE id = 10;
SELECT pg_sleep(10); -- Ждем 10 секунд
ROLLBACK;
```

Листинг 13 Скрипт Б (Окно 2, запускается сразу после А)

```
SELECT stars FROM hotel WHERE id = 10;
```



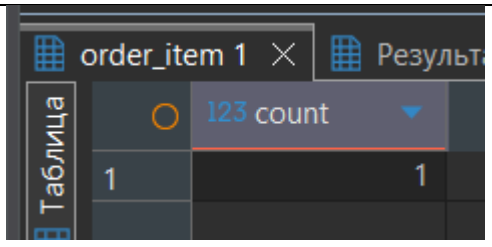
Моделирование аномалии «Фантомное чтение» (Phantom Read)

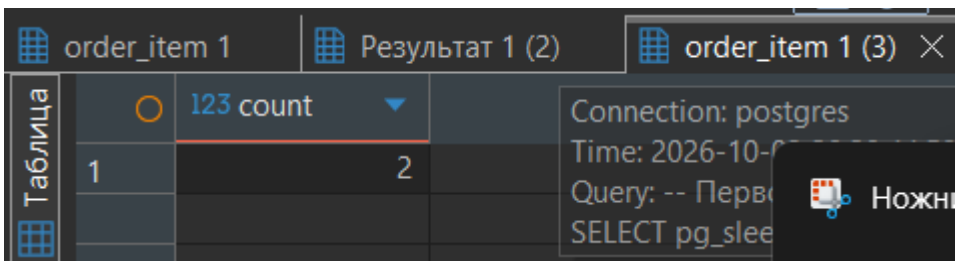
Листинг 14 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SELECT COUNT(*) FROM order_item WHERE order_id = 1; -- Первое чтение
SELECT pg_sleep(10);
SELECT COUNT(*) FROM order_item WHERE order_id = 1; -- Второе чтение
COMMIT;
```

Листинг 15 Скрипт Б, для выполнения во втором окне.

```
BEGIN;
INSERT INTO order_item (service_id, order_id, quantity, price)
VALUES (2, 1, 1, 10000);
COMMIT;
```

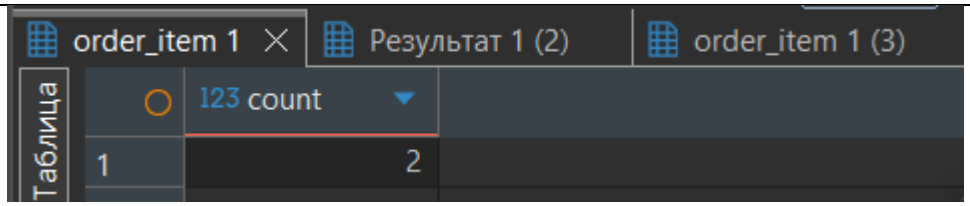


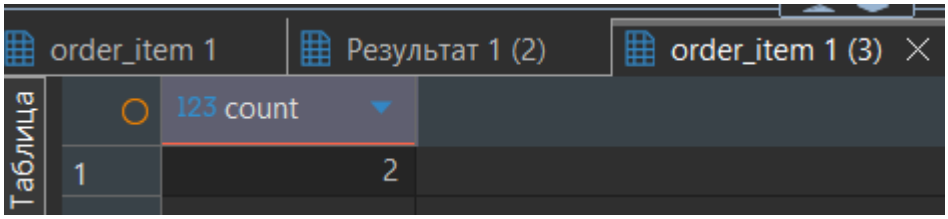


Устранение аномалии «Фантомное чтение» (Phantom Read)

Листинг 16 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM order_item WHERE order_id = 1;
SELECT pg_sleep(10);
SELECT COUNT(*) FROM order_item WHERE order_id = 1;
COMMIT;
```





ТРИГГЕРЫ И КУРСОРЫ В POSTGRESQL

Цель работы: Целью данной практической работы является формирование у студентов углубленных практических навыков по управлению данными и реализации сложной бизнес-логики в СУБД PostgreSQL с использованием триггеров и курсоров.

Задание №1: триггеры

Триггер 1: сложная валидация

Листинг 1

```
-- ============================================================
-- 0. Подготовка дополнительных объектов для триггеров
-- ============================================================

-- Таблица для аудита смены статусов заказов
CREATE TABLE IF NOT EXISTS order_status_history (
    id              serial PRIMARY KEY,
    order_id        integer NOT NULL,
    old_status_id   integer,
    new_status_id   integer NOT NULL,
    old_status_name varchar(50),
    new_status_name varchar(50) NOT NULL,
    changed_at      timestamptz NOT NULL DEFAULT now(),
    changed_by      text NOT NULL DEFAULT current_user
);

COMMENT ON TABLE order_status_history IS
    'История изменения статусов заказов клиентов';

COMMENT ON COLUMN order_status_history.order_id IS
    'ID заказа, по которому изменился статус';

COMMENT ON COLUMN order_status_history.old_status_id IS
    'Старый статус заказа до изменения';

COMMENT ON COLUMN order_status_history.new_status_id IS
    'Новый статус заказа после изменения';

COMMENT ON COLUMN order_status_history.changed_at IS
    'Дата и время фиксации изменения';

COMMENT ON COLUMN order_status_history.changed_by IS
    'Пользователь БД, от имени которого произошло изменение';

-- Индекс для быстрого поиска истории по заказу
CREATE INDEX IF NOT EXISTS idx_order_status_history_order_id
    ON order_status_history (order_id);
-- ============================================================
-- 1. Триггерная функция: проверка суммы возврата
-- ============================================================
```

```
CREATE OR REPLACE FUNCTION fn_validate_refund_amount()
RETURNS TRIGGER AS $$
DECLARE
    v_payment_amount   numeric(10,2);
    v_already_refunded numeric(10,2);
    v_available        numeric(10,2);
BEGIN
    -- Получаем сумму платежа
    SELECT amount
      INTO v_payment_amount
      FROM payment
     WHERE id = NEW.payment_id;

    IF v_payment_amount IS NULL THEN
        RAISE EXCEPTION 'Платеж с id=% не найден', NEW.payment_id;
    END IF;

    -- Считаем уже сделанные возвраты
    IF TG_OP = 'UPDATE' THEN
        -- При UPDATE исключаем текущую запись
        SELECT COALESCE(SUM(amount), 0)
          INTO v_already_refunded
          FROM refund
         WHERE payment_id = NEW.payment_id
           AND id <> NEW.id;
    ELSE
        -- При INSERT учитываем все существующие возвраты
        SELECT COALESCE(SUM(amount), 0)
          INTO v_already_refunded
          FROM refund
         WHERE payment_id = NEW.payment_id;
    END IF;

    -- Доступная сумма для возврата
    v_available := v_payment_amount - v_already_refunded;

    -- Дополнительная защита, хотя CHECK тоже существует
    IF NEW.amount <= 0 THEN
        RAISE EXCEPTION 'Сумма возврата должна быть положительной';
    END IF;

    -- Главная проверка
    IF NEW.amount > v_available THEN
        RAISE EXCEPTION
            'Превышена доступная сумма возврата. Платеж=%, сумма платежа=%,
уже возвращено=%, доступно=%, запрошено=%',
            NEW.payment_id,
            v_payment_amount,
            v_already_refunded,
            v_available,
            NEW.amount;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

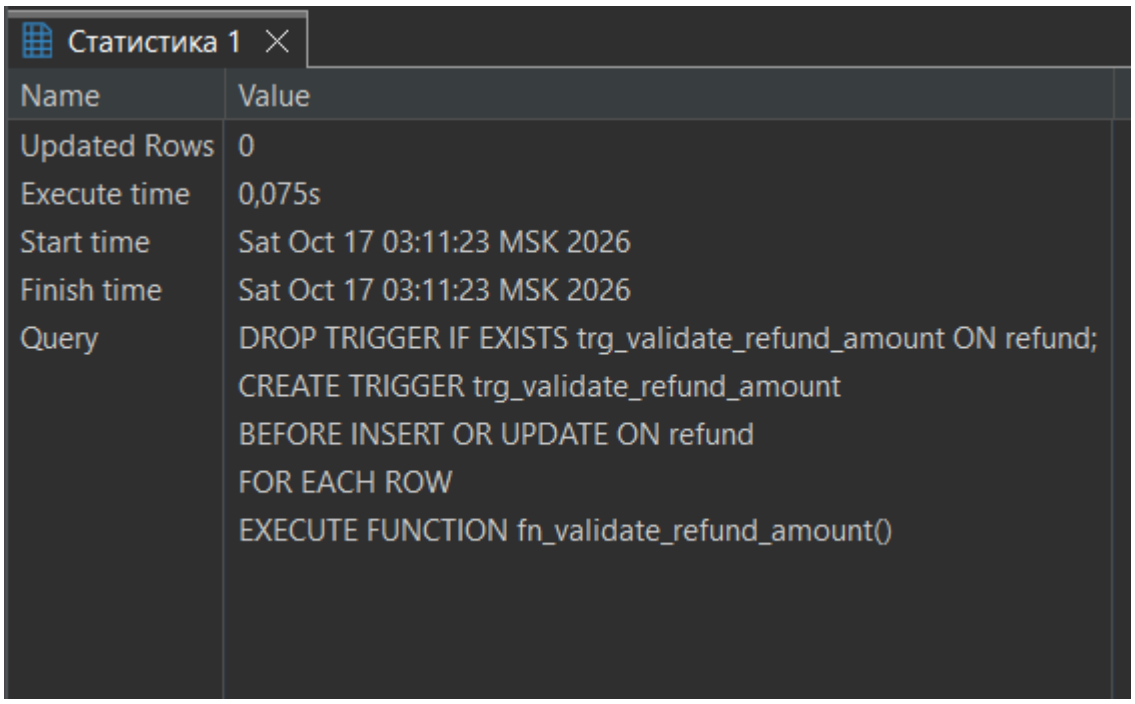
Name	Value
Updated Rows	0
Execute time	0,076s
Start time	Sat Oct 17 03:10:03 MSK 2026
Finish time	Sat Oct 17 03:10:03 MSK 2026

Query	CREATE OR REPLACE FUNCTION fn_validate_refund_amount()
	RETURNS TRIGGER AS $$
	DECLARE
	v_payment_amount numeric(10,2);
	v_already_refunded numeric(10,2);
	v_available numeric(10,2);
	BEGIN
	-- Получаем сумму платежа
	SELECT amount
	INTO v_payment_amount
	FROM payment
	WHERE id = NEW.payment_id;
	IF v_payment_amount IS NULL THEN
	RAISE EXCEPTION 'Платеж с id=% не найден', NEW.payment_id;
	END IF;
	-- Считаем уже сделанные возвраты
	IF TG_OP = 'UPDATE' THEN
	-- При UPDATE исключаем текущую запись
	SELECT COALESCE(SUM(amount), 0)
	INTO v_already_refunded
	FROM refund
	WHERE payment_id = NEW.payment_id
	AND id <> NEW.id;
	ELSE
	-- При INSERT учитываем все существующие возвраты
	SELECT COALESCE(SUM(amount), 0)
	INTO v_already_refunded
	FROM refund
	WHERE payment_id = NEW.payment_id;

Листинг 2

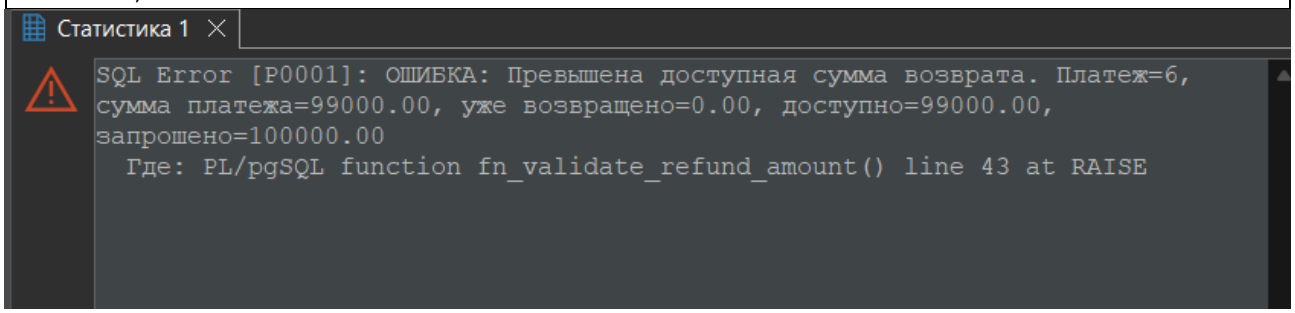
```
DROP TRIGGER IF EXISTS trg_validate_refund_amount ON refund;

CREATE TRIGGER trg_validate_refund_amount
BEFORE INSERT OR UPDATE ON refund
FOR EACH ROW
EXECUTE FUNCTION fn_validate_refund_amount();
```



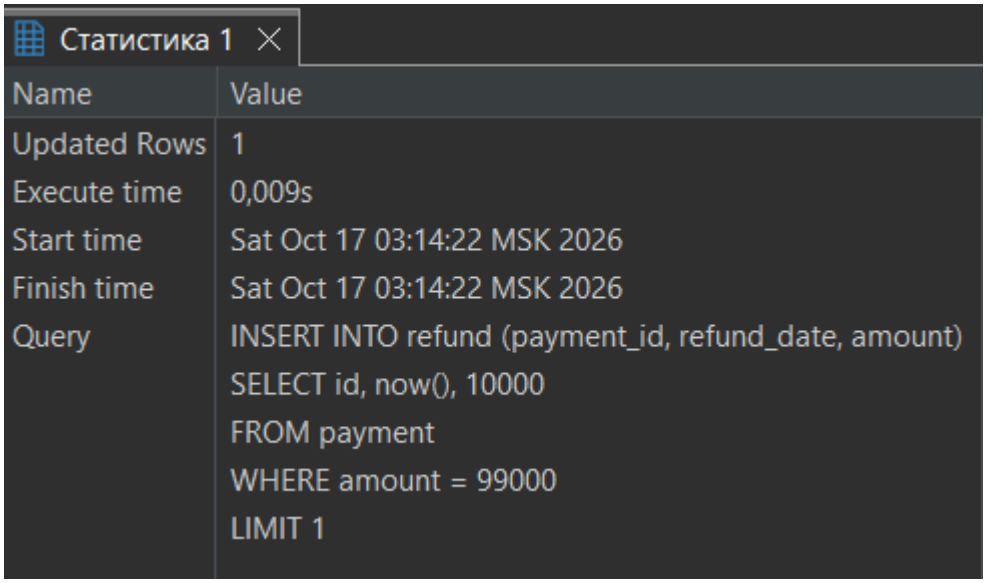
Листинг 3

```
INSERT INTO refund (payment_id, refund_date, amount)
SELECT id, now(), 100000
FROM payment
WHERE amount = 99000
LIMIT 1;
```



Листинг 4

```
INSERT INTO refund (payment_id, refund_date, amount)
SELECT id, now(), 10000
FROM payment
WHERE amount = 99000
LIMIT 1;
```



Триггер 2: поддержание согласованности

Листинг 5

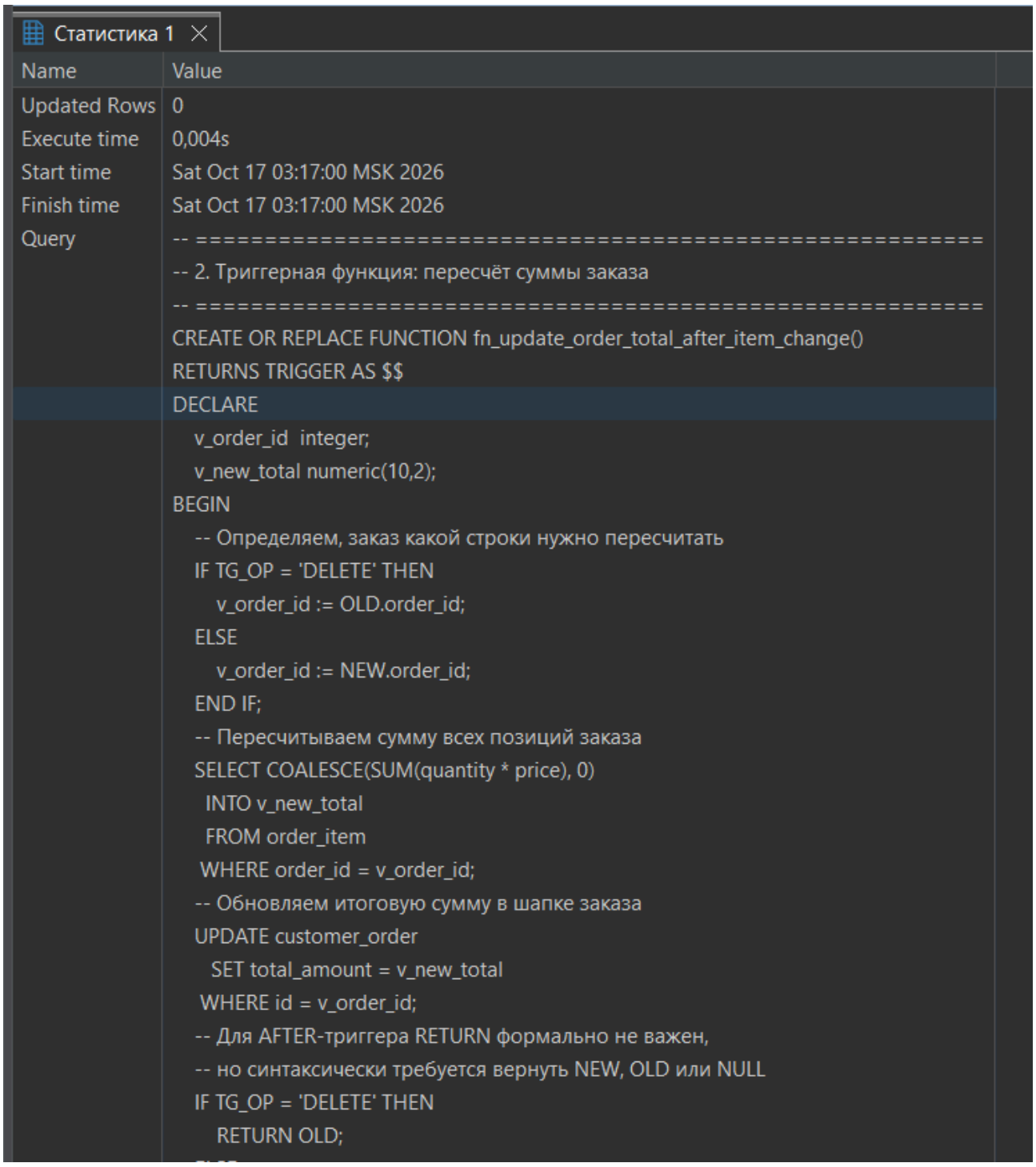
```
-- ============================================================
-- 2. Триггерная функция: пересчёт суммы заказа
-- ============================================================

CREATE OR REPLACE FUNCTION fn_update_order_total_after_item_change()
RETURNS TRIGGER AS $$
DECLARE
    v_order_id  integer;
    v_new_total numeric(10,2);
BEGIN
    -- Определяем, заказ какой строки нужно пересчитать
    IF TG_OP = 'DELETE' THEN
        v_order_id := OLD.order_id;
    ELSE
        v_order_id := NEW.order_id;
    END IF;

    -- Пересчитываем сумму всех позиций заказа
    SELECT COALESCE(SUM(quantity * price), 0)
      INTO v_new_total
      FROM order_item
     WHERE order_id = v_order_id;

    -- Обновляем итоговую сумму в шапке заказа
    UPDATE customer_order
       SET total_amount = v_new_total
     WHERE id = v_order_id;

    -- Для AFTER-триггера RETURN формально не важен,
    -- но синтаксически требуется вернуть NEW, OLD или NULL
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

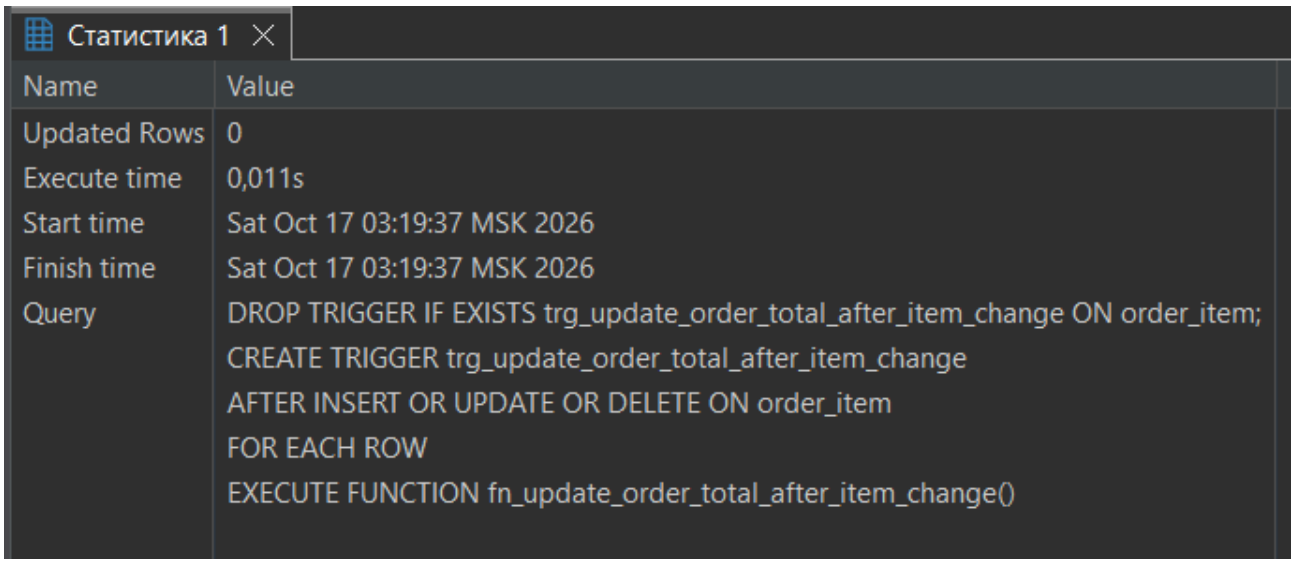


Статистика 1

Name	Value
Updated Rows	0
Execute time	0,004s
Start time	Sat Oct 17 03:17:00 MSK 2026
Finish time	Sat Oct 17 03:17:00 MSK 2026
Query	-- ==
	-- 2. Триггерная функция: пересчёт суммы заказа
	-- ==
	CREATE OR REPLACE FUNCTION fn_update_order_total_after_item_change()
	RETURNS TRIGGER AS $$
	DECLARE
	v_order_id integer;
	v_new_total numeric(10,2);
	BEGIN
	-- Определяем, заказ какой строки нужно пересчитать
	IF TG_OP = 'DELETE' THEN
	v_order_id := OLD.order_id;
	ELSE
	v_order_id := NEW.order_id;
	END IF;
	-- Пересчитываем сумму всех позиций заказа
	SELECT COALESCE(SUM(quantity * price), 0)
	INTO v_new_total
	FROM order_item
	WHERE order_id = v_order_id;
	-- Обновляем итоговую сумму в шапке заказа
	UPDATE customer_order
	SET total_amount = v_new_total
	WHERE id = v_order_id;
	-- Для AFTER-триггера RETURN формально не важен,
	-- но синтаксически требуется вернуть NEW, OLD или NULL
	IF TG_OP = 'DELETE' THEN
	RETURN OLD;

Листинг 6

```
DROP TRIGGER IF EXISTS trg_update_order_total_after_item_change ON order_item;
CREATE TRIGGER trg_update_order_total_after_item_change
AFTER INSERT OR UPDATE OR DELETE ON order_item
FOR EACH ROW
EXECUTE FUNCTION fn_update_order_total_after_item_change();
```



Листинг 7

```
-- ============================================================
-- Тест триггера 2: автоматический пересчёт total_amount
-- ============================================================

DO $$
DECLARE
    v_order_id   integer;
    v_service_id integer;
    v_total      numeric(10,2);
BEGIN
    -- Создаём тестовый заказ
    INSERT INTO customer_order (
        client_id,
        employee_id,
        status_id,
        paid_at,
        total_amount
    )
    VALUES (
        (SELECT id FROM client WHERE login = 'ivan_i'),
        (SELECT id FROM employee WHERE login = 'a_smirnov'),
        (SELECT id FROM order_status WHERE name = 'Новый'),
        NULL,
        0
    )
    RETURNING id INTO v_order_id;

    -- Берём существующую услугу
    SELECT id
      INTO v_service_id
      FROM service
     WHERE name = 'Тур в Анталию 7 дней';

    -- 1. INSERT позиции заказа
    INSERT INTO order_item (service_id, order_id, quantity, price)
    VALUES (v_service_id, v_order_id, 1, 10000);

    SELECT total_amount
      INTO v_total
      FROM customer_order
     WHERE id = v_order_id;

    RAISE NOTICE 'После INSERT: total_amount = % (ожидалось 10000)', v_total;
```

```
    -- 2. UPDATE позиции заказа
    UPDATE order_item
       SET quantity = 2
     WHERE order_id = v_order_id
       AND service_id = v_service_id;

    SELECT total_amount
      INTO v_total
      FROM customer_order
     WHERE id = v_order_id;

    RAISE NOTICE 'После UPDATE: total_amount = % (ожидалось 20000)', v_total;

    -- 3. DELETE позиции заказа
    DELETE FROM order_item
     WHERE order_id = v_order_id
       AND service_id = v_service_id;

    SELECT total_amount
      INTO v_total
      FROM customer_order
     WHERE id = v_order_id;

    RAISE NOTICE 'После DELETE: total_amount = % (ожидалось 0)', v_total;

    -- Удаляем тестовый заказ
    DELETE FROM customer_order
     WHERE id = v_order_id;

    RAISE NOTICE 'Тестовые данные удалены';
END $$;
```

```
После INSERT: total_amount = 10000.00 (ожидалось 10000)
После UPDATE: total_amount = 20000.00 (ожидалось 20000)
После DELETE: total_amount = 0.00 (ожидалось 0)
Тестовые данные удалены
```

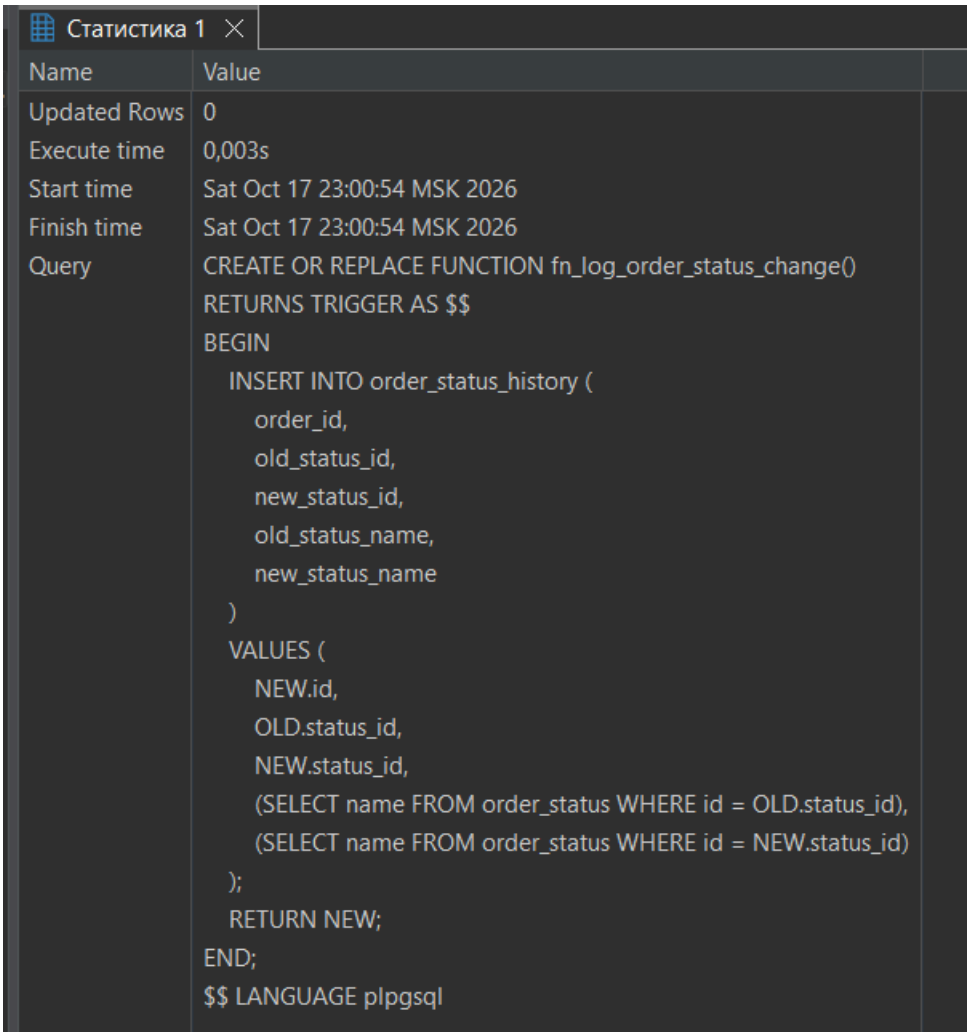
Триггер 3: аудит и логирование

Моделирование аномалии «Неповторяемое чтение»

Листинг 8.

```
CREATE OR REPLACE FUNCTION fn_log_order_status_change()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO order_status_history (
        order_id,
        old_status_id,
        new_status_id,
        old_status_name,
        new_status_name
    )
    VALUES (
        NEW.id,
        OLD.status_id,
        NEW.status_id,
        (SELECT name FROM order_status WHERE id = OLD.status_id),
        (SELECT name FROM order_status WHERE id = NEW.status_id)
    );
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```



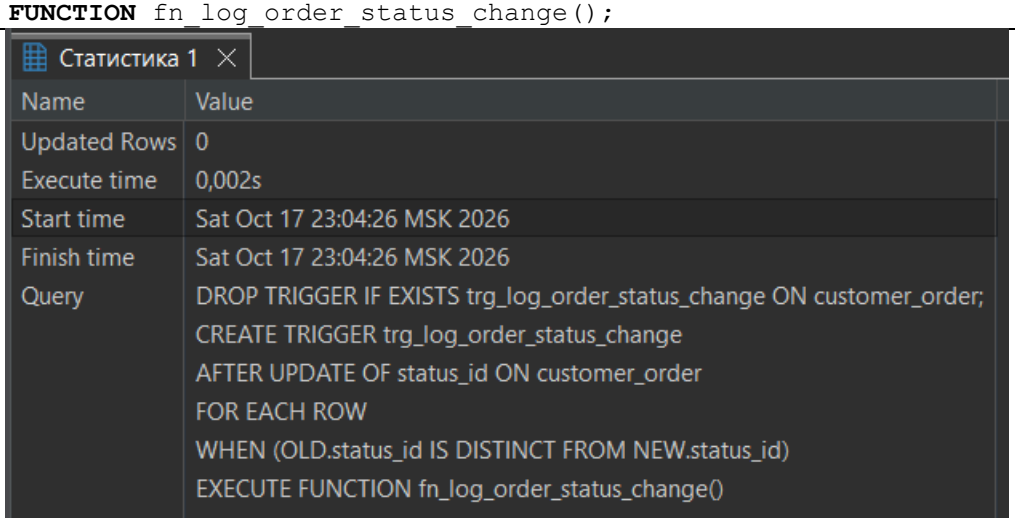
Статистика 1

Name	Value
Updated Rows	0
Execute time	0,003s
Start time	Sat Oct 17 23:00:54 MSK 2026
Finish time	Sat Oct 17 23:00:54 MSK 2026
Query	CREATE OR REPLACE FUNCTION fn_log_order_status_change() RETURNS TRIGGER AS $$ BEGIN INSERT INTO order_status_history (order_id, old_status_id, new_status_id, old_status_name, new_status_name) VALUES (NEW.id, OLD.status_id, NEW.status_id, (SELECT name FROM order_status WHERE id = OLD.status_id), (SELECT name FROM order_status WHERE id = NEW.status_id)); RETURN NEW; END; $$ LANGUAGE plpgsql

Листинг 9.

```
DROP TRIGGER IF EXISTS trg_log_order_status_change ON customer_order;

CREATE TRIGGER trg_log_order_status_change
AFTER UPDATE OF status_id ON customer_order
FOR EACH ROW
WHEN (OLD.status_id IS DISTINCT FROM NEW.status_id)
EXECUTE FUNCTION fn_log_order_status_change();
```



Статистика 1

Name	Value
Updated Rows	0
Execute time	0,002s
Start time	Sat Oct 17 23:04:26 MSK 2026
Finish time	Sat Oct 17 23:04:26 MSK 2026
Query	DROP TRIGGER IF EXISTS trg_log_order_status_change ON customer_order; CREATE TRIGGER trg_log_order_status_change AFTER UPDATE OF status_id ON customer_order FOR EACH ROW WHEN (OLD.status_id IS DISTINCT FROM NEW.status_id) EXECUTE FUNCTION fn_log_order_status_change()

Листинг 10

```
-- ============================================================
-- Тест триггера 3: аудит смены статуса заказа
-- ============================================================

DO $$
DECLARE
    v_order_id      integer;
    v_count_before  integer;
    v_count_after   integer;
BEGIN
    -- Создаём тестовый заказ со статусом "Новый"
    INSERT INTO customer_order (
        client_id,
        employee_id,
        status_id,
        paid_at,
        total_amount
    )
    VALUES (
        (SELECT id FROM client WHERE login = 'ivan_i'),
        (SELECT id FROM employee WHERE login = 'a_smirnov'),
        (SELECT id FROM order_status WHERE name = 'Новый'),
        NULL,
        12345
    )
    RETURNING id INTO v_order_id;

    -- Считаем записи в истории до изменения
    SELECT COUNT(*)
      INTO v_count_before
      FROM order_status_history;

    -- Меняем статус заказа
    UPDATE customer_order
       SET status_id = (
           SELECT id
           FROM order_status
           WHERE name = 'Подтверждён менеджером'
       )
     WHERE id = v_order_id;

    -- Считаем записи после изменения
    SELECT COUNT(*)
      INTO v_count_after
      FROM order_status_history;

    IF v_count_after = v_count_before + 1 THEN
        RAISE NOTICE 'Триггер аудита сработал корректно: добавлена 1 запись в
историю';
    ELSE
        RAISE NOTICE 'Ошибка: триггер аудита не сработал или сработал
некорректно';
    END IF;

    -- Запоминаем количество
    v_count_before := v_count_after;

    -- Обновляем заказ без смены статуса
    UPDATE customer_order
       SET total_amount = total_amount
     WHERE id = v_order_id;

    SELECT COUNT(*)
```

```
      INTO v_count_after
      FROM order_status_history;

    IF v_count_after = v_count_before THEN
        RAISE NOTICE 'Обновление без смены статуса не создало запись в истории
— корректно';
    ELSE
        RAISE NOTICE 'Ошибка: создана лишняя запись в истории';
    END IF;

    -- Просмотр последней записи истории для проверки
    RAISE NOTICE 'Последняя запись истории: %',
        (
            SELECT json_build_object(
                'order_id', h.order_id,
                'old_status', h.old_status_name,
                'new_status', h.new_status_name,
                'changed_by', h.changed_by
            )
            FROM order_status_history h
            WHERE h.order_id = v_order_id
            ORDER BY h.id DESC
            LIMIT 1
        );

    -- Очищаем тестовые данные
    DELETE FROM order_status_history
     WHERE order_id = v_order_id;

    DELETE FROM customer_order
     WHERE id = v_order_id;

    RAISE NOTICE 'Тестовые данные удалены';
END $$;
```

```
Триггер аудита сработал корректно: добавлена 1 запись в историю
Обновление без смены статуса не создало запись в истории — корректно
Последняя запись истории: {"order_id" : 19, "old_status" : "Новый", "new_status"
Тестовые данные удалены
```

Задание №2: курсоры

Явный курсор

Листинг 11

```
-- ============================================================
-- Явный курсор: построчный вывод заказов
-- ============================================================

DO $$
DECLARE
    -- Объявление курсора: связываем курсор с SQL-запросом
    cur_orders CURSOR FOR
        SELECT
            o.id AS order_id,
            c.login AS client_login,
            concat_ws(
                ' ',
```

```
                (c.full_name).last_name,
                (c.full_name).first_name,
                (c.full_name).middle_name
            ) AS client_fio,
            e.login AS employee_login,
            os.name AS status_name,
            o.total_amount
        FROM customer_order o
        JOIN client c ON c.id = o.client_id
        JOIN employee e ON e.id = o.employee_id
        JOIN order_status os ON os.id = o.status_id
        ORDER BY o.id;

    -- Переменная для текущей строки курсора
    r RECORD;
BEGIN
    -- Открытие курсора: выполняется запрос
    OPEN cur_orders;

    LOOP
        -- Извлечение следующей строки
        FETCH cur_orders INTO r;

        -- Если данных больше нет, выходим из цикла
        EXIT WHEN NOT FOUND;

        -- Обработка строки: вывод в консоль
        RAISE NOTICE
            'Заказ % | клиент: % (%) | менеджер: % | статус: % | сумма: %',
            r.order_id,
            r.client_login,
            r.client_fio,
            r.employee_login,
            r.status_name,
            r.total_amount;
    END LOOP;

    -- Закрытие курсора: освобождение ресурсов
    CLOSE cur_orders;
END $$;
```

```
Заказ 1 | клиент: ivan_i (Иванов Иван Иванович) | менеджер: a_smirnov | статус: Оплачен полностью | сумма: 95000.00
Заказ 2 | клиент: annap (Петрова Анна Сергеевна) | менеджер: al_popov | статус: Частично оплачен | сумма: 120000.00
Заказ 3 | клиент: alexs (Сидоров Алексей Владимирович) | менеджер: kat_lebed | статус: Оплачен полностью | сумма: 45000.00
Заказ 4 | клиент: lenak (Козлова Елена Дмитриевна) | менеджер: igor_nov | статус: Подтверждён менеджером | сумма: 210000.00
Заказ 5 | клиент: dmorozov (Морозов Дмитрий Алексеевич) | менеджер: a_smirnov | статус: Выкуплен у поставщика | сумма: 67000.00
Заказ 6 | клиент: olga_v (Волкова Ольга Николаевна) | менеджер: al_popov | статус: Подтверждён поставщиком | сумма: 99000.00
Заказ 7 | клиент: paveln (Никитин Павел Олегович) | менеджер: kat_lebed | статус: Отменён клиентом | сумма: 0.00
Заказ 8 | клиент: tat_s (Семёнова Татьяна Игоревна) | менеджер: igor_nov | статус: Оплачен полностью | сумма: 32000.00
Заказ 9 | клиент: artemg (Григорьев Артём Станиславович) | менеджер: al_popov | статус: Завершён | сумма: 366000.00
Заказ 10 | клиент: juliam (Михайлова Юлия Андреевна) | менеджер: a_smirnov | статус: Ожидает оплаты | сумма: 112000.00
```

Неявный курсор в функции – вывод результата в таблицу

Листинг 12 . Скрипт A (Окно 1)

```
-- ============================================================
-- Неявный курсор: функция, возвращающая таблицу
-- ============================================================

DROP FUNCTION IF EXISTS get_client_order_summary();

CREATE FUNCTION get_client_order_summary()
RETURNS TABLE (
    client_id_out  integer,
```

```
    full_name_out  text,
    email_out      varchar(50),
    orders_count   bigint,
    total_spent    numeric(10,2)
)
AS $$
DECLARE
    rec RECORD;
BEGIN
    -- Неявный курсор:
    -- PostgreSQL автоматически выполняет OPEN, FETCH, CLOSE
    FOR rec IN
        SELECT
            c.id AS client_id,
            concat_ws(
                ' ',
                (c.full_name).last_name,
                (c.full_name).first_name,
                (c.full_name).middle_name
            ) AS fio,
            c.email,
            COUNT(o.id) AS orders_count,
            COALESCE(SUM(o.total_amount), 0) AS total_spent
        FROM client c
        LEFT JOIN customer_order o ON o.client_id = c.id
        GROUP BY
            c.id,
            c.email,
            (c.full_name).last_name,
            (c.full_name).first_name,
            (c.full_name).middle_name
        ORDER BY c.id
    LOOP
        -- Присваиваем значения выходным полям функции
        client_id_out := rec.client_id;
        full_name_out := rec.fio;
        email_out     := rec.email;
        orders_count  := rec.orders_count;
        total_spent   := rec.total_spent;

        -- Добавляем текущую строку в итоговый табличный результат
        RETURN NEXT;
    END LOOP;

    -- Завершение функции
    RETURN;
END;
$$ LANGUAGE plpgsql;
```

Листинг 13 Скрипт Б (Окно 2, запускается сразу после А)

```
SELECT *
FROM get_client_order_summary();
```

SELECT * FROM get_client_order_summary() | Введите SQL выражение чтобы отфильтровать результаты

Таблица | Текст

	client_id_out	full_name_out	email_out	orders_count	total_spent
1	1	Иванов Иван Иванович	ivanov@mail.ru	1	95 000
2	2	Петрова Анна Сергеевна	anna_new@mail.ru	1	120 000
3	3	Сидоров Алексей Владимирович	sid_alex@yandex.ru	1	45 000
4	4	Козлова Елена Дмитриевна	lenakozlova@gmail.com	1	210 000
5	5	Морозов Дмитрий Алексеевич	morozov_d@bk.ru	1	67 000
6	6	Волкова Ольга Николаевна	volkova_o@inbox.ru	1	99 000